

Article

Tiny Language Model Guided Flow Q Learning for Optimal Task Scheduling in Fog Computing

Bhargavi K ^{1,*}  and Sajjan G. Shiva ^{2,*} ¹ Department of CSE, Siddaganga Institute of Technology, Tumakuru 572103, Karnataka, India² Department of CS, University of Memphis, Memphis, TN 38152, USA

* Correspondence: bhargavik@sit.ac.in (B.K.); sshiva@memphis.edu (S.G.S.)

Abstract

Fog computing is one of the rapidly growing platforms with an exponentially increasing demand for real-time data processing. The fog computing market is expected to reach USD 8358 million by the year 2030 with a compound annual growth of 50%. The wide adaptation of fog computing by the industries worldwide is due to the advantages like reduced latency, high operational efficiency, and high-level data privacy. The highly distributed and heterogeneous nature of fog computing leads to significant challenges related to resource management, data security, task scheduling, data privacy, and interoperability. The task typically represents a job generated by the IoT device. The action indicates the way of executing the tasks whose decision is taken by the scheduler. Task scheduling is one of the prominent issues in fog computing which includes the process of effectively scheduling the tasks among fog devices to effectively utilize the resources and meet the Quality of Service (QoS) requirements of the applications. Improper task scheduling leads to increased execution time, overutilization of resources, data loss, and poor scalability. Hence there is a need to do proper task scheduling to make optimal task distribution decisions in a highly dynamic resource-constrained heterogeneous fog computing environment. Flow Q learning (FQL) is a potential form of reinforcement learning algorithm which uses the flow matching policy for action distribution. It can handle complex forms of data and multimodal action distribution which make it suitable for the highly volatile fog computing environment. However, flow Q learning struggles to achieve a proper trade-off between the expressive flow model and a reduction in the Q function, as it relies on a one-step optimization policy that introduces bias into the estimated Q function value. The Tiny Language Model (TLM) is a significantly smaller form of a Large Language Model (LLM) which is designed to operate over the device-constrained environment. It can provide fair and systematic guidance to disproportionately biased deep learning models. In this paper a novel TLM guided flow Q learning framework is designed to address the task scheduling problem in fog computing. The neutrality and fine-tuning capability of the TLM is combined with the quick generable ability of the FQL algorithm. The framework is simulated using the Simcan2Fog simulator considering the dynamic nature of fog environment under finite and infinite resources. The performance is found to be good with respect to parameters like execution time, accuracy, response time, and latency. Further the results obtained are validated using the expected value analysis method which is found to be satisfactory.



Academic Editor: Frank Werner

Received: 9 December 2025

Revised: 7 January 2026

Accepted: 8 January 2026

Published: 10 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons](https://creativecommons.org/licenses/by/4.0/)[Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.**Keywords:** fog computing; tiny language model; flow Q learning; service placement; optimal solution

1. Introduction

Fog computing is a popular distributed computing environment which brings computation and storage closer to the fog nodes. The proximity of the data towards the fog nodes enhances the processing efficiency by reducing the latency and response time. There is an exponential growth in fog computing as the need for real-time data processing at fog nodes is high. The fog computing market is expected to reach 8358 million dollars by the year 2030 with a compound annual growth of 50%. It offers greater scalability by seamlessly adding the fog nodes at the edge network, and the resources are provided on a demand basis. Because of the distributed nature of the fog system, single point failures are prevented, and the system is resilient to attacks. The improved coordination among the fog nodes leads to quick data analysis at edge devices, location awareness, and improved overall efficiency of the fog services [1,2].

The dynamic nature typically represents time-specific varying characteristics of the fog computing system which affect the scheduling decisions. The tasks generated by the IoT devices vary continuously in unpredictable ways. The resource availability of fog nodes in terms of bandwidth, memory, and CPU utilization rate keeps fluctuating. Even the network conditions are not constant. The queue length keeps updating as the system state changes after every scheduling decision. The heterogeneity of the fog system represents unequal computation capability among the fog nodes. The energy, power, and latency across fog nodes also keep varying. Fog nodes are also spread across a wide geographic location, and every fog node cannot have a global and static view of the requirements. Hence it is not possible to formulate a single scheduling policy which is optimal for all heterogeneous task requests. Managing this uncertainty is challenging and demands the design and development of a fine-tuned machine learning model [3,4]. The machine learning model is mainly required to handle the high-dimensional, time-varying, and partially observable nature of fog computing. The simple heuristic solution is not feasible for fog computing as it is dependent on fixed-decision logic, limited awareness of fog states, and the static assumption of network characteristics. The machine learning model should understand system behavior and mine nonlinear relationships among fog nodes. The generic form of the machine learning algorithm suffers from poor exploration, slow convergence, and unstable learning over the large state space environment. The language model fine-tuned the machine learning algorithm by injecting the prior knowledge, including a guide during exploration, and accelerating the convergence process [5,6]. The fine-tuned machine learning models are considered as promising frameworks to address the performance issues in the fog environment like resource management, task scheduling, service placement, resource provisioning, and many more.

Out of all performance issues, task scheduling is considered as one of the most difficult tasks in an uncertain fog environment because of various factors like the arrival rate of tasks, fluctuating resource demands, and unpredictable network conditions. As the tasks are generated continuously, the workload demands fluctuate dynamically. The resource requirements of tasks generated by the heterogeneous IoT devices keeps fluctuating. The network congestion and failure of fog devices can cause significant damage to the performance of the fog system [5,6]. As a result, the chances of formulating the improper task scheduling decisions are high. Improper task scheduling policies are the policies that operate randomly by ignoring the current state of operation. This causes long-term side effects through violation of Service Level Agreements (SLAs) and inefficient usage of resources. Various kinds of improper task scheduling happen that include random assignment, greedy assignment, threshold-based assignment, and round robin assignment. In random assignment, the task is randomly assigned to the computing fog node without taking into consideration current load and latency sensitivity of the tasks. In greedy assignment, all

incoming tasks are assigned to the nearest fog node intentionally even though that node becomes overloaded and other nodes are underloaded. In threshold-based assignments, a fixed threshold value is predetermined based on which decisions are made despite the fluctuations in the network and task requirements. In round robin assignment, the tasks are distributed among the fog nodes sequentially with fixed time slots for processing, in which the heterogeneous nature of the tasks and resources are completely neglected. Hence there is a need to develop a task scheduler which can handle uncertainties and formulate high quality task scheduling policies [7].

In literature, several classical heuristic algorithms and machine learning algorithms are developed for task scheduling in fog environments. Heuristic algorithms are dependent on the fixed assumptions which fail to operate in a real-time fog computing environment. The task's arrival pattern is usually bursty and resources availability changes continuously. The predetermined rules do not learn from experience and lack online adaptation. They perform manual fine tuning of parameters and assume all tasks have identical QoS requirements. As the complexity of the rules increases there is a communication overhead and parameter fine tuning becomes infeasible. The partial observability and uncertain execution pattern of the fog environment weakens the scheduler performance. Hence, the learning base approaches the need to be designed to learn through experience and make long-term decisions through adaptability. Machine learning models directly capture nonlinear interaction between tasks, fog nodes, and network. Through trial and feedback mechanisms the machine learning algorithm becomes tolerant to uncertainty and gains the ability to generalize from experience [8].

Flow Q learning (FQL) is one of the promising offline forms of reinforcement learning techniques which models complex action distributions using flow matching policy. The existing actor critic framework is updated using flow matching policy and does not require any form of sampling strategy. The flow policies are basic, and maximum value is reached using recursive backpropagation. The core idea is to use a one-step policy along with the flow policy with behavioral cloning to maximize the Q value. The performance of the FQL is better than the Gaussian policies as it uses weighted regression, recursive backpropagation, and rejection sampling. However, FQL performance suffers because of overestimation of action values and is very sensitive to hyperparameter values. It also struggles to achieve a proper trade-off between exploration and exploitation. The size of the Q table grows exponentially and takes a longer time to converge to an optimal solution due to slow updates of Q values. It also struggles with poor reward shaping and heuristics that are not available for sampling the state and action pairs [9,10]. It becomes difficult for the FQL algorithm to learn from scratch about complex dynamics of the fog computing environment, so Tiny Language Models (TLMs) provide guidance to FQL in a structured manner which accelerates the speed to learning and stabilizes it quickly.

TLMs are potential lightweight alternatives to Large Language Models (LLMs). The main motivation to select a TLM is that it can operate in a resource-constrained environment. It also offers low inference overhead and allows for near-edge deployment. It is made up of fewer parameters which are properly fine-tuned and can deliver high-level performance using strategies such as byte-level tokenization, pooling, and weight training. The TLMs are better compared to generic Large Language Models, deep neural networks, heuristic models, and centralized cloud approaches. The generic language models include Large Language Model Meta AI (LLaMA), Generative Pre-trained Transformer (GPT), and Bidirectional Encoder Representations from Transformers (BERT). The deep neural networks include Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), and Q learning. Heuristic approaches include the Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO). Centralized cloud approaches in-

clude deep Q network (DQN), Software-Defined Network (SDN), and Kubernetes. The heuristics of TLM act as source of guidance for the flow Q learning algorithm. The sampling efficiency of the FQL is enhanced at every step of training, which in turn provides the desired policies in subsequent iteration. It also extends support for the online and offline corrections of heuristics which prevents biased performance in a dynamic fog environment. The heuristics of TLM speed up the learning capability of the FQL agent and recover seamlessly from the incorrect guidance [11,12].

In this paper a novel TLM guided FQL framework is designed for the fog computing environment. The FQL enriched with the heuristics of the TLM can formulate optimal task scheduling policies even from imperfect data by considering the suggestions from generative artificial intelligence models. Immediate shaping of Q values is carried out according to the heuristics of TLM which enhances the reward shaping mechanism [13]. The unnecessary explorations over the state space are prevented through pre-sampling and post-sampling of the state space under the guidelines of the TLM [14,15].

The objectives of the paper are as follows:

- A brief introduction to fog computing and discussing the factors causing uncertainties which influence the task scheduler performance.
- Enriching the traditional FQL framework with the heuristics of the TLM to formulate task scheduling policies which are free from bias within a few epochs of training.
- Mathematical representation of the fog computing system model with the definitions for the performance metrics considered for evaluation purpose.
- Performing a literature survey over the recent existing works and summarizing the research gaps identified.
- Design of a novel TLM guided FQL framework to perform task scheduling.
- Expected value analysis of the TLM guided FQL framework considering finite and infinite fog computing scenarios.
- Simulation of the TLM guided FQL framework using Simcan2Fog simulator considering real-time and synthetic workloads.

The remaining sections of the paper are organized as follows: Section 2 discusses the related work, Section 3 gives the system model for fog computing, Section 4 discusses the proposed TLM guided flow Q framework, Section 5 performs expected value analysis of the proposed framework, Section 6 discusses the result and discussion, and finally Section 7 ends with the conclusions.

2. Related Work

In this section some of the potential recent papers on task scheduling for fog computing are discussed and their limitations are identified.

Choppara et al. presents an adaptive form of the task scheduling framework for fog computing by combining the federated deep Q network with the K-means clustering algorithm [16]. Each of the fog nodes in the fog environment is made capable enough for training the local deep Q network model. The experience of each of the fog nodes will be merged to formulate a global model without the need to share any sensitive information. The federated nature of learning provides a self-optimizing solution which evenly spreads the learning among all fog nodes. The priorities are assigned for the incoming tasks considering the two parameters of execution time and deadline for completion using the K-means clustering algorithm. Care is always taken such that higher priority tasks are executed by minimizing the rejection rate and Service Level Agreement violations. The model continuously upgrades itself and adapts to a dynamically changing environment through iterative learning. The localized form of learning incorporates constant learning among local deep Q learning agents which constantly update the task scheduling policies

considering both task and environment characteristics. The experimental evaluation over the larger dataset ensures that makespan time is reduced and throughput is enhanced. Since distributed task scheduling is performed, the framework acts as a flexible solution for latency-sensitive applications. For a larger network like fog, decentralization themed task scheduling enhances the system scalability and reduces the communication overhead but demands for high synchronization. Prioritized experience replay is performed in which higher priorities enable the model to focus over the most informative experience. However, the framework demands frequent updates from the sender and receiver models which makes it expensive. The deeply nested Q network often causes instability of the network due to frequent updating of the Q value and higher sampling efficiency over the large state space environment.

Choppara et al. discuss a flexible task scheduling framework based on the deep deterministic form of policy gradient (DDPG) for fog computing [17]. The DDPG is dynamically adaptable for continuous and large state space environments in which the task scheduling policies adapt themselves according to real-time demands of the tasks in terms of resources. The DDPG basically follows the actor critic learning model in which the actor learns the policy and the critic evaluates the action taken using the Q function value. The experimental results reveal that satisfactory performance is achieved by reducing the makespan time and increasing the fault tolerance. The data processing operation is localized which reduces the bandwidth requirement and enhances the efficiency of data management across the fog nodes. The priority score is calculated for each of the tasks considering the key parameters like resource requirement, duration of execution, task deadline, and dependency for the tasks. Critical applications and delay-sensitive applications are executed first based on the priority score. After prioritization, the tasks are classified into three categories which include sequential tasks, parallel tasks, and dependency-oriented tasks. For sequential tasks, an execution order will be defined which is well-suited for data pipeline applications. The parallel tasks are executed on a certain set of virtual machines which are mounted over the fog nodes. For dependency-based tasks, logical consistency is ensured in the workflow which demands to satisfy the specific requirements prior to execution. However, the framework is subjected to a divergence problem due to high variance encountered in the estimation of the Q value. Also, a tendency to end up in local optima is high due to high sensitivity towards hyperparameters and sparse reward in the deterministic fog computing environment.

Sudheer et al. discuss a methodology for task scheduling in fog computing by combining priority awareness approach with the simulated annealing mechanism [18]. The simulated annealing algorithm begins with the initialization of the state space with sub optimal solutions. Later the optimality of the solution will be improved by decreasing the value of the temperature variable. The random task plan is assigned for each of the fog nodes at first. Afterwards a small tweak in the task plan is performed by considering the parameters like energy consumption, cost of execution, completion time of task, and priority assigned for the tasks. This leads to the generation of neighbors and a controlled exploration of the state space is performed. The small tweak performed affects only two workloads: the old node which was executing the workload and the new node to which the workload was transferred. Initially the temperature is set to be high, but the algorithm slowly reduces the temperature value through the hierarchy to converge with the best potential solution in a fine-grained approach. The scheduling policies are improved gradually by computing the cost for each of the small changes in an iterative manner. During the annealing phase, higher weightage is given for exploring the new solutions instead of revisiting the existing solutions. The movement is carried out in an uphill manner by accepting all the worst forms of solutions which allow the exploration of state space in a wider manner. The proper

trade-off is achieved between exploration and exploitation through tough optimization process. During the cooling phase, the gradual cooling of temperature is achieved to arrive at an ideal solution while keeping the search area open wide enough for traversal. Finally, algorithms stop when the temperature falls below the threshold or the k number of iterations are completed. The performance results show that the high priority tasks are processed early using a priority aware penalty function. The solution space is approached in a very strict and controlled manner this prevents large shifts leading to a local optimal solution. However, the algorithm is completely stochastic in nature which leads to different kinds of outcome at every run. The cooling process is slow and requires more epochs of training compared to the traditional spring embedders technique.

Asad et al. presents a novel task scheduling framework for a cloud and fog environment using the Harris Hawk Optimization (HHO) technique [19]. Task scheduling is an important issue as it is difficult to find appropriate resources for the execution of the tasks. Scheduling the client task to fog nodes means tasks can be executed with less delay but it deals with more energy consumption. Meanwhile scheduling the tasks onto a cloud server consumes less energy but more delay is incurred. Hence it is essential for the task scheduling algorithm to consider all the factors including energy and delay before arriving at scheduling decisions. The HHO technique mimics the cooperative hunting strategy of the Harris Hawk. It mainly works in two phases: exploration and exploitation. During the exploration phase the location of prey is located by the Hawk. Similarly, during the exploitation phase, all possible attack strategies are executed over the prey. At first the arrival rate of tasks and energy consumed for escaping of prey are initialized. The task is assigned to fog nodes based on the factors like the Quality of Service (QoS) requirements of tasks and location of the data source. The diversity in the available solution is determined using inverse mutation operation. The best solution out of all available solutions is stored in the repository as the non-dominated solution. Through epochs of training the new solution replaces the old solution based on the value of crowding distance. The soft besiege phase represents the start of the attacking strategy in which the energy level of the prey is depleted using the encircling mechanism. Similarly, the hard besiege phase performs encircling of the prey in a zigzag manner. The deep dive operation of the Hawks is performed using the Levy flight procedure. Whenever the prey becomes fatigued an effective attack will be launched to execute the surprise pounce. The process is repeated until termination is reached and the best solution available at the search agent is considered as the optimal scheduling policy. The experimental results indicate that there is a significant reduction in energy consumption and delay. Guaranteed performance is achieved by achieving proper balance between intensification and diversification phases. However, random parameters are involved in the exploration and exploitation phases which cause negative effects over the convergence speed and lead to higher computational cost.

Seyed et al. perform the scheduling of a bunch of tasks in fog computing using the discrete cuckoo search algorithm [20]. First the incoming client applications are broken down into many independent tasks. A centralized broker node is maintained which is responsible for maintaining the capacity of fog nodes along with their failure rates. The broker node is responsible for performing load balancing and allocating the task onto the most reliable fog node. The myriad permutation policy is followed by delivery of different varieties of solutions to the incoming client requests. The typical cuckoo search algorithm is converted into hybrid form by incorporating two procedures related to sorting and crowding distance computation. Numerous local search operators are used to perform exhaustive search over the solution space which consists of the worst solution and best solution. The discrete search space of the fog is explored towards the efficient form of quest permutations. The diverse novel exploration operation is invoked randomly using the

Levy flight search mechanism. The workflow is typically represented as a directed acyclic graph; the exploration and exploitation operators are executed during the encoding stage. The topological ordering of the tasks is determined to preserve the constraints among the tasks. The main aim of the optimization is to generate new capable solutions over an iteration of training. If the new solution generated is better than the previous solution, then the new one is considered as the dominant solution. Here multiple objectives are taken into consideration while delivering the solution. The solutions are divided into two sets that are an inferior set of solutions and superior set of solutions using Pareto ranking values. The superior set of solutions focuses on performing multiple objectives optimization. Whereas the inferior set of solutions focuses on finding the single objective optimization. The set of solutions which do not compete are assigned to first rank and known as non-dominated solutions. The first rank solutions are removed from the current solutions, and the procedure is repeated for the remaining solutions. Likewise, second and third ranking solutions are represented. All solutions generated are sorted based on the number of objectives and crowding distance value. The solution with higher crowding distance value is considered as beneficial because it exhibits a high rate of diversity. The experimental results showcase that significant improvement is observed with respect to reliability and makespan time in all varying scenarios of workload conditions. However, here the scheduler is not aware of the user's Quality of Service requirements, which increases the monetary cost of operation and the scheduler fails to achieve the desired QoS.

Seohang et al. discussed the basic working of flow Q learning algorithm [21]. It projects the flow matching policy to represent the complex action distribution in a large state space environment. The flow model is based on the stochastic differential equation which allows for faster training and converges with better quality solutions. Three important benefits offered by flow Q learning are as follows: It uses a reparametrized policy gradient which perform effective exploration of state space and prevents the unstable and expensive backpropagation mechanism. Second, the one-step policy is generated as the output which overcomes the inherent limitations of iterative flow generation process. Third, the implementation is easy and fine tuning is performed at optimal pace. It is composed of only one major hyperparameter and does not demand fine tuning in a noisy environment. However, flow Q learning suffers from training instability and poor generalization capability under dynamic scenario. It consists of many sensitive parameters which need to be fine-tuned and exhibits poor scalability due to the composition of a large input space. To overcome exploration inefficiency and solve the credit assignment problem, poor guidance is required.

Fuxue et al. discuss the need for LLM guidance to the traditional reinforcement learning algorithm [22]. The reinforcement learning algorithm cannot perform high-level planning and low-level task execution simultaneously. The LLM is responsible for decomposing larger tasks into a sequence of manageable subtasks in an interactive state space heterogeneous environment. During training even the intermediate form of subgoals are associated with the rewards and fine-grained feedback is provided to accelerate the convergence rate and perform structured exploration of the large state space environment. The chain of thought strategy is incorporated which allows the LLM to quickly update to changing dynamics of the environment state space. The exploration inefficiency in sparse reward setting is overcome by conditioning the subgoals and reward functions. The LLM guided Q learning operates using generic prompts which prevent frequent retraining of the model. The reward is scaled as per the completion of subgoals, and more rewards are obtained through faster scaling. However, LLM-enabled reinforcement learning algorithms cannot be deployed for the fog environment, as LLMs are composed of billions of parame-

ters and demand a large amount of memory. The output of LLM is often hallucinated and even invalid which makes it incompatible with the larger scale edge deployments scenarios.

To summarize, most of the existing works exhibit the following drawbacks:

- The deep neural network is often stuck in learning instability over a large state space environment which leads to learning instability and converges with non-desirable policies [16].
- The deterministic policy gradients generated by the Q learning agent are extremely sensitive towards the selection of hyperparameters. They also suffer from sparse reward problems [17].
- Metaheuristic algorithms are extremely costly over the distributed form of a heterogeneous environment. Quick scheduling decisions are taken using heuristic values which leads to errors in task scheduling decisions [19,20].
- The rule-based algorithms usually fail to capture the uncertainty of the devices, workload, and network condition. Converges to suboptimal solutions due to improper balance between the exploration and exploitation activities [3].
- Supervised and unsupervised machine learning algorithms suffer from poor search progress and weaker efficiency. They are often subjected to overfitting problems and suffer from generalization issues due to high variance and learning instability [5,6].
- Conventional flow Q learning algorithms suffer from credit assignment problems and poor exploration capability [21].
- Traditional LLM guided reinforcement learning is subjected to the sparse reward problem and struggles to perform multistage scheduling in long horizons. Often ends up with invalid outputs and inefficient utilization of resources [22].

3. System Model

The fog computing system model is composed of a set of tasks $T = \{T_1, T_2, T_3, T_4, \dots, T_m\}$ and a set of fog nodes $FN = \{FN_1, FN_2, FN_3, FN_4, \dots, FN_n\}$. Each fog node is made up of a collection of virtual machines $VM = \{VM_1, VM_2, VM_3, VM_4, \dots, VM_k\}$. Each of the tasks is described using five attributes: task duration TD , resource requirements RR , deadline proximity DP , dependency count DC , and impact factor IF , i.e., $T_i = \langle TD, RR, DP, DC, IF \rangle$. The TD is the estimated execution time of the tasks on any fog node. The RR is the computational resource requirement of the task to be executed. The DP indicates the deadline sensitivity of the task. The DC represents the number of tasks that are dependent on the execution of the current task. The impact factor IF is the composite measure of the severity of the poor scheduling decisions. For example: For healthcare application a sample attributes for a particular task includes $T_i = \langle TD = 10ms, RR = 200 \text{ CPUs}, DP = \text{Less}, DC = 3 \text{ Tasks}, IF = \text{High} \rangle$. For a small execution time, the IF value must be lesser for critical applications like healthcare and military in which improper scheduling results in severe consequences and ends up in poor Quality of Service (QoS). Similarly, each of the virtual machines represents the current workload $C(WL)$ and capacity of the CPU, $CP(CPU)$, for all virtual machines in the fog node, i.e., $VM_i = \langle C(WL), CP(CPU) \rangle$. The task scheduling problem is typically represented as a Markov Decision Process (MDP), $MDP(T_i) = \langle S, A, P, R, \gamma \rangle$, where S is the state space, A is the action space, P is the transition probability, R is the reward function, and γ is the discount factor which achieves balance between the immediate reward and future reward. The description for the attributes of MDP are shown in Table 1.

Table 1. MDP attributes description.

State space $S = \{S_t^{T_i}, S_t^{FN_i}, S_t^{net_i}\}$	The state space represents the state space of tasks $S_t^{T_i}$, state space of fog nodes $S_t^{FN_i}$, and state space of fog network $S_t^{net_i}$. • $S_t^{T_i} = \{\text{Queue length, task size, task deadline, task priority}\}$ • $S_t^{FN_i} = \{\text{CPU utilization, Memory, Energy consumption, processing power}\}$ • $S_t^{net_i} = \{\text{Transmission delay, bandwidth availability}\}$
Action space $A = \{a_t^{assign}\}, a_t^{assign} \rightarrow FN_i$	Assigning the incoming tasks to a fog node
Transition probability $P(S' S, a) = Prb(S_{t+1} = s' s_t = s, A_t = a)$	Probability of transition from state S to state S' after action a.
Reward $R = -(w_1 D_t + w_2 E_t + w_3 P_t)$	Scalar reward represents the multiple objective optimization encoding process. • $w_1, w_2, w_3 \geq 0$, Weights assigned • D_t = Task completion delay • E_t = Energy consumption of the fog node • P_t = Penalty for SLA violation
Discount factor γ	Discount factor value ranges between 0 and 1, i.e., $\gamma \in [0, 1]$

The incoming tasks are arranged in a queue; the arrival and departure rate of the tasks are managed by a task scheduler. During FQL the training of flow policy is performed using a one-step policy for arbitrarily complex data distributed across the fog domain. Direct guiding of iterative flow is not performed instead the value of the one-step policy is maximized through distilling the flow model. As a result, the unstable recursive backpropagation is prevented which eliminates the need for costly action generation in the iterative mode. All the last optimal task scheduling policies are generated $\Pi = \langle \Pi_1, \Pi_2, \Pi_3, \dots, \Pi_p \rangle$.

The mathematical definition for the evaluation metrics (EM) considered for evaluation of the framework is defined as follows. The reason behind the selection of these evaluation metrics is that they are the benchmark metrics followed worldwide for fog deployment. They basically represent the main objectives and constraints of a fog computing environment. In which the task requirements are dynamic, resources are less, but still there is a need to provide best Quality of Service (QoS). Cost metrics represent the economic efficiency of the task scheduler. Energy consumption metric is a critical evaluation metric because the scheduler needs to operate in a resource-constrained environment. SLA violation rate is important for delay-sensitive applications to meet the deadline and latency constraints. Scalability indicates the improved performance of the scheduler with the increasing number of tasks and fog nodes. Reliability is the ability of the scheduler to function even under fog nodes failure and network disruptions [23].

EM1: Cost (CO(TLM_FQL)): The cost of the TLM_FQL is the summation of the cost incurred in the computation $CP(T_i)$, communication $CM(T_i)$, and usage of resources $UR(T_i)$ of tasks:

$$CO(TLM_FQL) = \sum_{i=1}^{i=m} (CP(T_i) + CM(T_i) + UR(T_i)) \quad (1)$$

where $CP(T_i) = CO(CPU_usage) * ET(T_i)$, $CO(CPU_usage)$ is the cost of CPU usage per unit of time in the fog node, and $ET(T_i)$ is the execution time of the tasks in the fog node. $CM(T_i) = CO(Mem_usage) * Mem(T_i)$, $CO(Mem_usage)$ is the cost of memory usage per data unit in the fog node, $Mem(T_i)$ is the memory required for the execution of the task T_i . $UR(T_i) = CO(BW_usage) * BW(T_i)$, $CO(BW_usage)$ is the cost of bandwidth usage per data unit in the fog node, and $BW(T_i)$ is the bandwidth allocated for the execution of the task T_i .

EM2: Energy consumption (EC(TLM_FQL)): The energy consumption of the TLM_FQL is the summation of the energy consumed for computation of the task $EC_{comp}(T_i)$ and energy consumed for transmission of the task $EC_{Tr}(T_i)$:

$$EC(TLM_FQL) = \sum_{i=1}^{i=m} (EC_{comp}(T_i) + EC_{Tr}(T_i)) \quad (2)$$

where $EC_{comp}(T_i) = \alpha_i(comp) * C_i * CPU_freq(T_i)^2$, $\alpha_i(comp)$ is the energy consumption of the task T_i during computation, C_i is the constant size of computation, and $CPU_freq(T_i)$ is the CPU clock frequency of the task T_i . $EC_{Tr}(T_i) = \frac{\alpha_i(tran)*S_i}{R_{Tr}}$, $\alpha_i(tran)$ is the energy consumption of the task T_i during transmission, S_i is the constant size of transmission, and R_{Tr} is the rate of transmission.

EM3: Service Level Agreement (SLA) violation rate (SLAVR(TLM_FQL)): This is the measure of frequency with which the service provider fails to meet the SLA with the clients. It is calculated by measuring the number of tasks which fail to complete within the predefined deadline:

$$SLAVR(TLM_FQL) = \frac{1}{N} \sum_{i=1}^{i=m} (IF(CT(T_i) > DL(T_i))) \quad (3)$$

where IF is the indicator function whose value range between $[0, 1]$. IF returns 1 whenever SLA violation happens and 0 otherwise. $CT(T_i) = ST(T_i) + DU(T_i)$ is the completion time of task, $ST(T_i)$ is the start time of task, and $DU(T_i)$ is the duration of task. $DL(T_i) = ST(T_i) - Due_T(T_i)$ is the deadline of the task and $Due_T(T_i)$ is the due time of the task.

EM4: Scalability (SC(TLM_FQL)): The scalability of TLM_FQL is the ability of the framework to effectively handle the increase in the number of incoming tasks. The framework effectively scales to adapt to dynamic changes without a drastic drop in performance.

$$SC(TLM_FQL) = \frac{MK(\sum_{i=1}^{i=m} T_i + \sum_{i=1}^{i=m} \Delta T_i) - MK(\sum_{i=1}^{i=m} T_i)}{\sum_{i=1}^{i=m} \Delta T_i} \quad (4)$$

where $MK(\sum_{i=1}^{i=m} T_i)$ is the makespan time of the task T_i and $MK(\sum_{i=1}^{i=m} T_i + \sum_{i=1}^{i=m} \Delta T_i)$ is the makespan time after adding ΔT_i number of task. The ideal value of $SC(TLM_FQL)$ is the one in which the makespan time is minimum with the increase in the number of tasks.

EM5: Reliability (RE(TLM_FQL)): Reliability of the TLM_FQL is ability of the framework to successfully operate for a predefined period without failure. It ensures that it operates consistently by efficiently handling data processing and data transmission without issues. Higher resource utilization indicates that the fog node is effectively utilized which optimizes the overall system performance.

$$RE(TLM_FQL) = \sum_{j=1}^{j=n} \frac{\sum_{i=1}^{i=m} Ex_time(T_i)}{Av_time(FN_i)} \quad (5)$$

where $Ex_time(T_i)$ is the execution time of the task, and $Av_time(FN_i)$ is the available time of the fog node.

4. Proposed Work

The high-level architecture of TLM guided FQL framework is shown in Figure 1. The architecture is composed of two main components that are the Tiny Language Model (TLM) and flow Q learning (FQL). TLM is a mini version of the Large Language Model (LLM) which has few parameters and can perform with lesser computational resources. The training of TLM is performed by a pre-trained model through knowledge distillation. Pruning is performed to remove unnecessary parts of the model and model weights are

updated through quantization. Deploying TLM on edge devices allows ease of interaction among edge devices, analysis, and localization of the data. It begins with the preparation of its own data through model pretraining. The model is built as an open GPT-2 basic model whose parameters vary from 30 M to 124 M. Fine tuning of the data is performed through downstream embedding of the sensed tasks. The trained model is converted into an optimized model to provide guidance to the FQL.

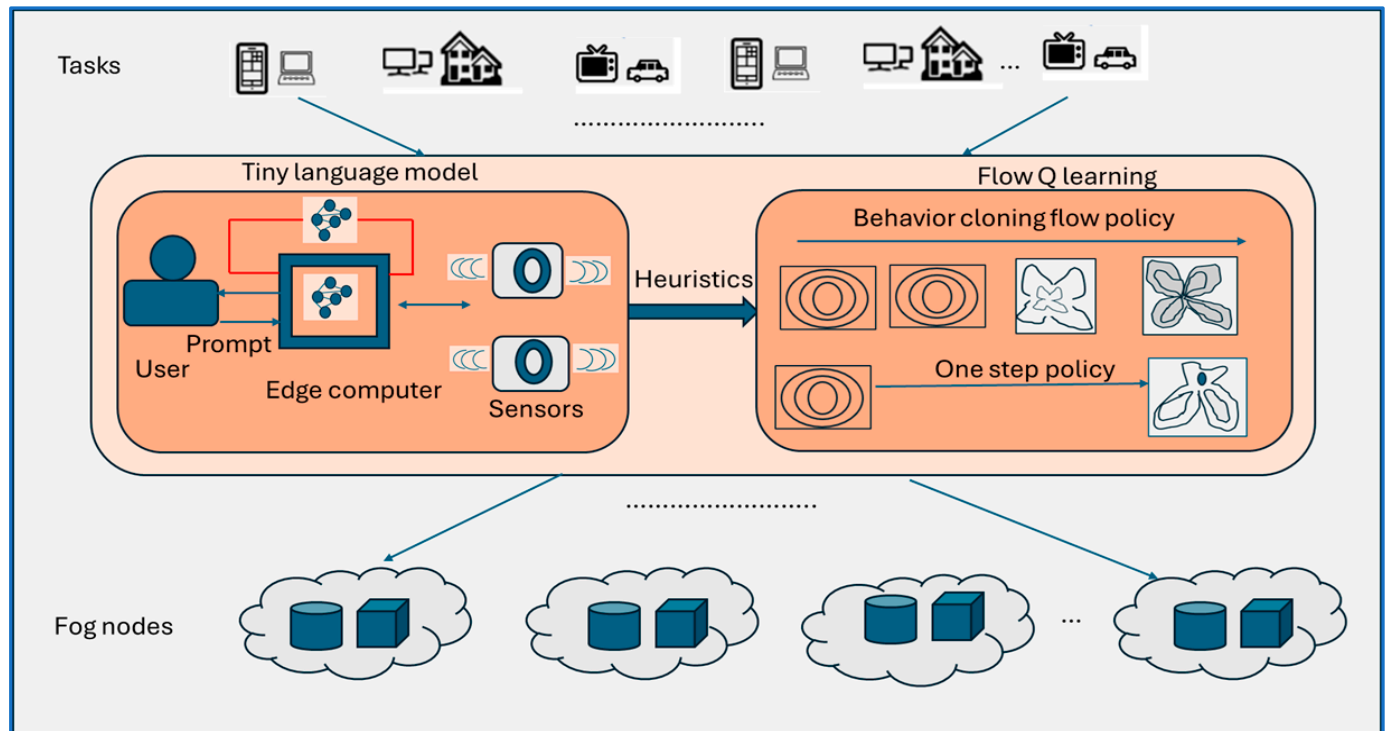


Figure 1. The proposed high-level architecture of the TLM guided FQL framework.

The flowchart representation of the TLM guided FQL is given in Figure 2. The FQL considers the heuristics of TLM to generate flow policies through precise modeling of the complex action distributions in the fog domain. Instead of performing random initialization of the parameters of the Q learning agent, the TLM are prompted to generate state, action, and Q values based on the high-level description of the tasks and fog nodes. This provides a promising starting point for the agent and only significant heuristics are added to the Q value update function which leads to execution of the best action at any given state of operation. The diffusion policies are trained to maximize the learned value function of the Q learning agent. behavior cloning is a form of supervised learning which trains the Q learning agent by mimicking the expert demonstrations. The sampling efficiency is improved through behavioral cloning, and it is also less subjected to compounding errors. Traditional Q learning algorithms are trained using multi-step policies which suffer from performance degradation as it is highly sensitive towards the number of steps chosen. This problem is overcome here using a one-step policy which enables direct one-step action prediction from the state through the flow matching mechanism. Direct mapping is performed from state to action, and the necessity of a separate iterative process is eliminated by the learning velocity field which maps noise to action. The performance of the FQL is higher compared to the Gaussian model and diffusion model as it can easily learn multimodal action distributions in a complex environment. Even the poor heuristics of the TLM are handled effectively by the Q learning agent through environment feedback and it is ensured that the task scheduling policies are unbiased and optimal.

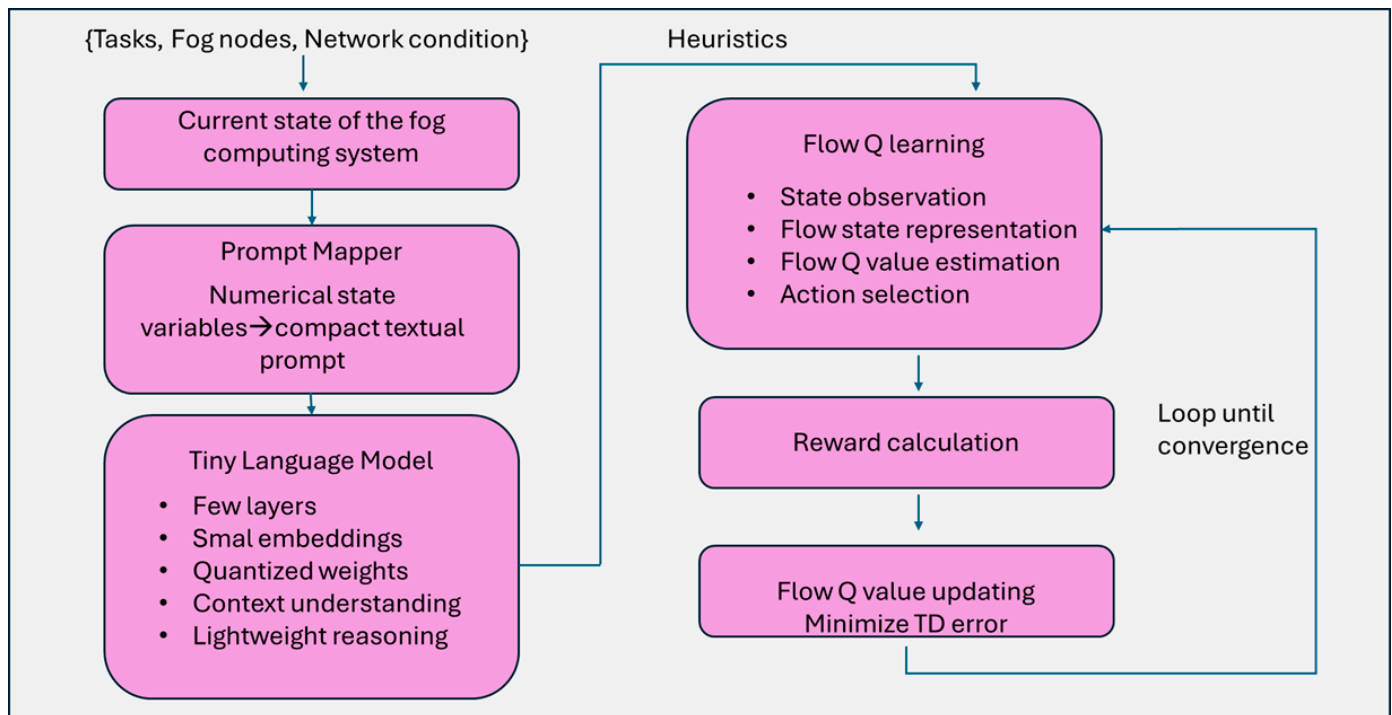


Figure 2. Flowchart representation of TLM guided FQL framework.

Algorithm 1 provides detailed discussion on the workings of TLM guided FQL framework.

The fog computing system state space is defined by five attributes, i.e., $FC_SS = \{R_{FN}, Q_{FN}, E_{FN}, L_{FN}, T_{FN}\}$, R_{FN} is the resources at the fog nodes, Q_{FN} is the queue length at each of the fog node, E_{FN} is the energy of the fog node, L_{FN} is the network latency, and T_{FN} is the attributes of the tasks. The concrete scheduling action is taken for each of the tasks, $a_i = \{Execute, Offload\ to\ cloud, Delay/reject\}$, by considering the heuristic scores of the actions generated by TLM. The input for TLM includes the characteristics of tasks, fog nodes, and the network condition. The state to encoder converts the numerical input into textual tokens. For example, if CPU utilization is greater than 80 percent it is considered as high load. TLM outputs a small set of action preference tokens. The deterministic action mapping subroutine is responsible for converting the tokens of TLN into heuristic scores. The heuristic score ranges between -0.5 and $+1.0$.

The FQL is mainly composed of three components, i.e., critic $Q_{\emptyset}(s, a)$, behavior cloning flow policy $\mu_{\theta}(s, z)$, and one-step policy $\mu_w(s, z)$. The critic basically provides learning signal via a gradient target for the flow policy generated by the agent. Bellman consistency is ensured by the critic by precisely estimating the future returns. The behavior cloning flow policy trains the flow policy to mimic the action of the dataset and it becomes the starting point of the agent. It also teaches the flow model to map noise to action. After performing behavior cloning the Q learning agent improves itself beyond the initialized dataset. The one-step policy takes the action immediately from a state without the need for any iterations. However, multiple steps of transformation are executed within to select a particular action as the optimal one. Using one-step policy the TLM_FQL parses through the tasks set by considering only the present Q state. This makes it greedy and leads to immediate scheduling decisions with multi-step planning.

Algorithm 1. Working of TLM guided FQL framework

```

1: Begin
2: Input:  $T = \{T_1, T_2, T_3, T_4, \dots, T_m\}$ ,  $FN = \{FN_1, FN_2, FN_3, FN_4, \dots, FN_n\}$ ,
    $VM = \{VM_1, VM_2, VM_3, VM_4, \dots, VM_k\}$ , Tiny Language Model with set of
   actions  $TLM(S) \rightarrow \{a_1, a_2, a_3\}$ , Q learning agent  $Q(s, a, r, s') \cong D$ ,  $D$  = transition
   dataset,  $s$  = state of environment,  $a$  = action in the state,  $r$  = reward received after
   taking action  $a$  in state  $s$ ,  $s'$  = next state after taking action  $a$ .
3: Output:  $\Pi = \langle \Pi_1, \Pi_2, \Pi_3, \dots, \Pi_p \rangle$ 
4:   Training of TLM_FQL
5:   for every training task  $T_i$  in  $T$  do
6:     Behavior cloning flow policy
7:     function: Policy induced by the flow model, i.e.,  $\mu_\theta(s, Z)$ , where  $s$  = state space,
        $Z$  = sampled noise
8:     for  $t = 0, 1, \dots, M - 1$  do
9:       Eulers methods approximate ordinary differential equation in the action space
        $\frac{dz_t}{dt} A = v_\theta(t, s, z_t)$ , where  $t$  is the time interval  $t \in [0, 1]$ ,  $z_t$  is the current value of
       the ordinary differential equation
10:      Perform Euler discretization with step size of  $1/M$ 
11:       $z_t = z_t + v_\theta(\frac{t}{M}, s, Z) / M$ ,  $v_\theta$  is the vector field
12:      Return  $z_t$ 
13:    End for
14:    While not converged to optimal solution do
15:      Sample the batch  $(s, a, r, s') \cong D$ 
16:      Train the critic of the TLM_FQL  $Q_\phi(SLM\_FQL)$ 
17:      Determine sampled value from normal Gaussian distribution
18:       $z = \mathcal{N}(0, I_d)$ ,  $\mathcal{N}$  = Gaussian distribution,  $I_d$  is the covariance identity matrix
19:      Critic evaluate the actions chosen by TLM  $Q_\phi(s, a_1)$ ,  $Q_\phi(s, a_2)$ , and  $Q_\phi(s, a_3)$ 
20:      Critic select the highest value action out of all the actions suggested by TLM
        $a' = \operatorname{argmax}_a Q_\phi(s, a)$ 
21:      Perform highest value action suggested by TLM  $a'$ , i.e.,  $a' = \mu_w(s', z)$ ,  $\mu_w$  is the
       flow-based policy considering next state  $s'$  and latent noise  $z$ 
22:      Update all learnable parameters of the critic  $Q_\phi(SLM\_FQL)$ 
23:      Minimize  $E[Q_\phi(s, a) - r - \gamma Q_\phi(s', a')]^2$ 
24:      Train vector field  $v_\theta$  in the behavior cloning flow policy  $\pi_\theta$ 
25:       $x^0 \cong \mathcal{N}(0, I_d)$ 
26:       $x^1 \leftarrow a$ 
27:       $t = \operatorname{Unif}[0, 1]$ , where  $\operatorname{Unif}$  is the uniform distribution, its value ranges
       between 0 and 1
28:       $x^t = (1 - t)x^0 + tx^1$ ,  $x^t$  is the loss function, linear interpolation  $x^0$  and  $x^1$ 
29:      Update  $\theta$  to minimize loss function  $L(\theta) = E[v_\theta(t, s, x^t) - (x^1 - x^0)]^2_2$ 
30:      Train one-step policy  $\pi_w$ 
31:       $z = \mathcal{N}(0, I_d)$ 
32:       $a^\pi = \mu_w(s, z)$ 
33:      Update  $w$  to minimize  $E[-Q_\phi(s, a^\pi) + \alpha \|a^\pi - \mu_\theta(s, z)\|^2_2]$ 
34:    End while
35:    Return one-step policy  $\pi_w$ 
36:  End for
37:  Formulate the task scheduling policies:  $\Pi ::= \Pi \cup \Pi_w$ 

```

Algorithm 1. *Cont.*

```

38: Testing of TLM_FQL
39: for every testing task  $T_i$  in  $T$  do
40:   While not converged to optimal solution do
41:     Test the critic of the TLM_FQL  $Q_{\phi}(SLM\_FQL)$ 
42:     Execute the highest value action suggested by TLM  $a' = \operatorname{argmax}_a Q_{\phi}(s, a)$ 
43:     Perform sampled value from normal Gaussian distribution  $z = \mathcal{N}(0, I_d)$ 
44:     Minimize  $E[Q_{\phi}(s, a) - r - \gamma Q_{\phi}(s', a')^2]$ 
45:     Test vector field  $v_{\theta}$  in the behavior cloning flow policy  $\pi_{\theta}$ 
46:     Update  $\theta$  to minimize loss function  $L(\theta) = E[v_{\theta}(t, s, x^t) - (x^1 - x^0)]_2^2$ 
47:     Test one-step policy  $\pi_w$ 
48:     Update  $w$  to minimize  $E[-Q_{\phi}(s, a^{\pi}) + \alpha \|a^{\pi} - \mu_{\theta}(s, z)\|_2^2]$ 
49:   End While
50:   Return one-step policy  $\pi_w$ 
51: End for
52: Formulate the task scheduling policies:  $\Pi ::= \Pi \cup \Pi_w$ 
53: Output the optimal task scheduling policies, i.e.,  $\Pi = \langle \Pi_1, \Pi_2, \dots, \Pi_p \rangle$ 
54: End

```

5. Expected Value Analysis

The expected value analysis of the TLM_FQL is performed considering the small scenario, medium scenario, and big scenario [24,25]. Table 2 provides a unified description for all the symbols used for expected value analysis.

Table 2. Description of notations used for expected value analysis.

Small Scenario	
Task set	$T = \{T_1, T_2, T_3, T_4, \dots, T_m\}$
Fog nodes set	$FN = \{FN_1, FN_2, FN_3, FN_4, \dots, FN_n\}$
Limited number of task scheduling polices	$\Pi = \{\Pi_1, \Pi_2, \Pi_3, \Pi_4, \dots, \Pi_p\}$.
Time intervals	$\langle \Delta T - \alpha \rangle, \langle \Delta T \rangle$, and $\langle \Delta T + \alpha \rangle$
Integral limit of evaluation metric over the finite policy set	$\int_x^y \frac{EMi}{\Pi = \{\Pi_1, \Pi_2, \Pi_3, \Pi_4, \dots, \Pi_p\}}$
Big Scenario	
Task set	$T = \{T_1, T_2, T_3, T_4, \dots, T_{\infty}\}$
Fog nodes set	$FN = \{FN_1, FN_2, FN_3, FN_4, \dots, FN_{\infty}\}$
Unlimited number of tasks scheduling polices	$\Pi = \{\Pi_1, \Pi_2, \Pi_3, \Pi_4, \dots, \Pi_{\infty}\}$.
Integral limit of evaluation metric over the infinite policy set	$\int_{-\infty}^0 \frac{EMi}{\Pi = \{\Pi_1, \Pi_2, \Pi_3, \Pi_4, \dots, \Pi_{\infty}\}} + \int_0^{+\infty} \frac{EMi}{\Pi = \{\Pi_1, \Pi_2, \Pi_3, \Pi_4, \dots, \Pi_{\infty}\}}$

5.1. Small Scenario

The small scenario is composed of a small number of tasks $T = \{T_1, T_2, T_3, T_4, \dots, T_m\}$, small number of fog nodes $FN = \{FN_1, FN_2, FN_3, FN_4, \dots, FN_n\}$, and a limited number of scheduling policies $\Pi = \{\pi_1, \pi_2, \pi_3, \pi_4, \dots, \pi_p\}$. The expected outcome of the analysis during three continuous time intervals, i.e., $\langle \Delta T - \alpha \rangle, \langle \Delta T \rangle$, and $\langle \Delta T + \alpha \rangle$, are

generated. It is the estimated likelihood of each evaluation metric which is found to be either low or high, $EO = < Low \text{ or } High >$.

EM1: Cost (CO(TLM_FQL)): The expected value of CO(TLM_FQL) is influenced by the expected value of the cost incurred in the computation $CP(T_i)$, communication $CM(T_i)$, and usage of resources $UR(T_i)$ of tasks.

$$EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T\right) = \int_x^y \frac{\sum_{i=1}^m EV(CP(T_i) + CM(T_i) + UR(T_i))}{|\Pi|} \quad (6)$$

$$EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T\right) = \int_x^y EV\left(\frac{CP(T_i)}{\Pi}, \Delta T\right) + EV\left(\frac{CM(T_i)}{\Pi}, \Delta T\right) + EV\left(\frac{UR(T_i)}{\Pi}, \Delta T\right) \quad (7)$$

$$EV\left(\frac{CP(T_i)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{CO(CPU_usage) * ET(T_i)}{\Pi} = \frac{EV(1 * \Pi * CP(T_i))}{\Pi} = \int_q^Q Q * dP * \frac{CP(T_i)}{\Pi} \quad (8)$$

$$EV\left(\frac{CM(T_i)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{CO(Mem_usage) * Mem(T_i)}{\Pi} = \frac{EV(1 * \Pi * CM(T_i))}{\Pi} = \int_q^Q Q * dP * \frac{CM(T_i)}{\Pi} \quad (9)$$

$$EV\left(\frac{UR(T_i)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{CO(BW_usage) * BW(T_i)}{\Pi} = \frac{EV(1 * \Pi * UR(T_i))}{\Pi} = \int_q^Q Q * dP * \frac{UR(T_i)}{\Pi} \quad (10)$$

$$EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) = Low, EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T\right) = Low, EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) = Low \quad (11)$$

EM2: Energy consumption (EC(TLM_FQL)): The expected value of EC(TLM_FQL) is influenced by the expected value of the energy consumed for the computation of the task $EC_comp(T_i)$ and energy consumed for transmission of the task $EC_Tr(T_i)$.

$$EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T\right) = \int_x^y \frac{\sum_{i=1}^m EV(EC_comp(T_i) + EC_Tr(T_i))}{|\Pi|} \quad (12)$$

$$EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T\right) = \int_x^y EV\left(\frac{EC_comp(T_i)}{\Pi}, \Delta T\right) + EV\left(\frac{EC_Tr(T_i)}{\Pi}, \Delta T\right) \quad (13)$$

$$EV\left(\frac{EC_comp(T_i)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{\alpha_i(comp) * C_i * CPU_freq(T_i)^2}{\Pi} = \frac{EV(1 * \Pi * EC_comp(T_i))}{\Pi} = \int_q^Q Q * dP * \frac{EC_comp(T_i)}{\Pi} \quad (14)$$

$$EV\left(\frac{EC_Tr(T_i)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{\frac{\alpha_i(tran) * S_i}{R_Tr}}{\Pi} = \frac{EV(1 * \Pi * EC_Tr(T_i))}{\Pi} = \int_q^Q Q * dP * \frac{EC_Tr(T_i)}{\Pi} \quad (15)$$

$$EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) = Low, EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T\right) = Low, EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) = Low \quad (16)$$

EM3: Service Level Agreement (SLA) violation rate (SLAVR(TLM_FQL)): The expected value of EC(TLM_FQL) is influenced by the expected value of the indicator function of the completion time of task $CT(T_i)$ and is the deadline of the task $DL(T_i)$.

$$EV\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T\right) = \int_x^y \frac{\frac{1}{N} \sum_{i=1}^m EV(IF(CT(T_i)) + EV(DL(T_i)))}{|\Pi|} \quad (17)$$

$$EV\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T\right) = \int_x^y EV\left(\frac{IF(CT(T_i))}{\Pi}, \Delta T\right) + EV\left(\frac{IF(EV(DL(T_i)))}{\Pi}, \Delta T\right) \quad (18)$$

$$EV\left(\frac{IF(CT(T_i))}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{ST(T_i) + DU(T_i)}{\Pi} = \frac{EV(1 * \Pi * IF(CT(T_i)))}{\Pi} = \int_q^Q Q * dP * \frac{IF(CT(T_i))}{\Pi} \quad (19)$$

$$EV\left(\frac{IF(DL(T_i))}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{ST(T_i) - Due_T(T_i)}{\Pi} = \frac{EV(1 * \Pi * IF(DL(T_i)))}{\Pi} = \int_q^Q Q * dP * \frac{IF(DL(T_i))}{\Pi} \quad (20)$$

$$\begin{aligned} EV\left(\frac{SLAVR(TLM_{FQL})}{\Pi}, \Delta T - \alpha\right) &= Low, EV\left(\frac{SLAVR(TLM_{FQL})}{\Pi}, \Delta T\right) \\ &= Low, EV\left(\frac{SLAVR(TLM_{FQL})}{\Pi}, \Delta T + \alpha\right) = Low \end{aligned} \quad (21)$$

EM4: Scalability (SC(TLM_FQL)): The expected value of SC(TLM_FQL) is influenced by the expected value of the makespan time of the task T_i $MK(\sum_{i=1}^m T_i)$ and makespan time after adding ΔT_i $MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i)$.

$$EV\left(\frac{SC(TLM_{FQL})}{\Pi}, \Delta T\right) = \int_x^y \frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i) - MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i} \frac{1}{|\Pi|} \quad (22)$$

$$EV\left(\frac{SC(TLM_{FQL})}{\Pi}, \Delta T\right) = \int_x^y EV\left(\frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i)}{\sum_{i=1}^m \Delta T_i}, \Delta T\right) + EV\left(\frac{MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i}, \Delta T\right) \quad (23)$$

$$\begin{aligned} &= EV\left(\frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i)}{\sum_{i=1}^m \Delta T_i}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i)}{\sum_{i=1}^m \Delta T_i} = \frac{EV\left(1 * \Pi * \frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i)}{\sum_{i=1}^m \Delta T_i}\right)}{\Pi} = \\ &\int_q^Q Q * dP * \frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i)}{\sum_{i=1}^m \Delta T_i} \end{aligned} \quad (24)$$

$$EV\left(\frac{MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i}, \Delta T\right) = \sum_{d \in D} d \int_x^y \frac{MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i} = \frac{EV\left(1 * \Pi * \frac{MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i}\right)}{\Pi} = \int_q^Q Q * dP * \frac{MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i} \quad (25)$$

$$\begin{aligned} EV\left(\frac{SC(TLM_{FQL})}{\Pi}, \Delta T - \alpha\right) &= High, EV\left(\frac{SC(TLM_{FQL})}{\Pi}, \Delta T\right) = High, \\ EV\left(\frac{SC(TLM_{FQL})}{\Pi}, \Delta T + \alpha\right) &= High \end{aligned} \quad (26)$$

EM5: Reliability (RE(TLM_FQL)): The expected value of SC(TLM_FQL) is influenced by the expected value of the execution time of the task $Ex_time(T_i)$ and available time of the fog node $Av_time(FN_i)$.

$$EV\left(\frac{RE(TLM_{FQL})}{\Pi}, \Delta T\right) = \int_x^y \sum_{j=1}^{j=n} \frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)} \frac{1}{|\Pi|} \quad (27)$$

$$EV\left(\frac{RE(TLM_{FQL})}{\Pi}, \Delta T\right) = \int_x^y EV\left(\sum_{j=1}^{j=n} \frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)}, \Delta T\right) \frac{1}{|\Pi|} \quad (28)$$

$$EV\left(\frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)}, \Delta T\right) = \sum_{d \in D} d \int_x^y \sum_{j=1}^{j=n} \frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)} \frac{1}{|\Pi|} = \frac{EV\left(1 * \pi * \frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)}\right)}{\Pi} = \int_q^Q Q * dP * \frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)} \quad (29)$$

$$EV\left(\frac{RE(TLM_{FQL})}{\Pi}, \Delta T - \alpha\right) = High, EV\left(\frac{RE(TLM_{FQL})}{\Pi}, \Delta T\right) = High, EV\left(\frac{RE(TLM_{FQL})}{\Pi}, \Delta T + \alpha\right) = High \quad (30)$$

5.2. Big Scenario

The big scenario is composed of an infinite number of tasks $T = \{T_1, T_2, T_3, T_4, \dots, T_\infty\}$, small number of fog nodes $FN = \{FN_1, FN_2, FN_3, FN_4, \dots, FN_\infty\}$, and infinite number of scheduling policies $\Pi = \{\Pi_1, \Pi_2, \Pi_3, \Pi_4, \dots, \Pi_\infty\}$. The expected outcome of the analysis during three continuous time intervals, i.e., $< \Delta T - \alpha >$, $< \Delta T >$, and $< \Delta T + \alpha >$, are generated. It is the estimated likelihood of each evaluation metric which is found to be either low or high, $EO = < Low \text{ or } High >$.

EM1: Cost (CO(TLM_FQL)): The expected value of the cost incurred by TLM_FQL in the big scenario is aligned with the expected value of the cost incurred by TLM_FQL in small scenario.

$$EV^*\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T\right) = \int_{-\infty}^0 \frac{\sum_{i=1}^m EV(CP(T_i) + CM(T_i) + UR(T_i))}{|\Pi|} + \int_0^{+\infty} \frac{\sum_{i=1}^m EV(CP(T_i) + CM(T_i) + UR(T_i))}{|\Pi|} \quad (31)$$

$$EV^*\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_{-\infty}^0 \frac{\sum_{a \in \pi} CO(TLM_FQL)(a)}{|\Pi|} + \int_0^{+\infty} \frac{\sum_{a \in \pi} CO(TLM_FQL)(a)}{|\Pi|} \quad (32)$$

$$EV^*\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong Low \geq EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong Low \quad (33)$$

$$EV^*\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T\right) \cong Low \rightarrow EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T\right) \cong Low \quad (34)$$

$$EV^*\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong Low \geq EV\left(\frac{CO(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong Low \quad (35)$$

EM2: Energy consumption (EC(TLM_FQL)): The expected value of the energy consumed by TLM_FQL in the big scenario is very much the same as the expected value of the energy consumed by TLM_FQL in small scenario.

$$EV^*\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T\right) = \int_{-\infty}^0 \frac{\sum_{i=1}^m EV(EC_comp(T_i) + EC_Tr(T_i))}{|\Pi|} + \int_0^{+\infty} \frac{\sum_{i=1}^m EV(EC_comp(T_i) + EC_Tr(T_i))}{|\Pi|} \quad (36)$$

$$EV^*\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_{-\infty}^0 \frac{\sum_{a \in \pi} EC(TLM_FQL)(a)}{|\Pi|} + \int_0^{+\infty} \frac{\sum_{a \in \pi} EC(TLM_FQL)(a)}{|\Pi|} \quad (37)$$

$$EV^*\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong Low \rightarrow EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong Low \quad (38)$$

$$EV^*\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T\right) \cong Low \rightarrow EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T\right) \cong Low \quad (39)$$

$$EV^*\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong Low \geq EV\left(\frac{EC(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong Low \quad (40)$$

EM3: Service Level Agreement (SLA) violation rate (SLAVR(TLM_FQL)): The expected value of the SLA violate rate of the TLM_FQL is significantly less compared to the expected value of the SLA violate rate of the TLM_FQL in small scenario.

$$EV^*\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T\right) = \int_{-\infty}^0 \frac{\frac{1}{N} \sum_{i=1}^m EV(IF(CT(T_i)) + EV(DL(T_i)))}{|\Pi|} + \int_0^{+\infty} \frac{\frac{1}{N} \sum_{i=1}^m EV(IF(CT(T_i)) + EV(DL(T_i)))}{|\Pi|} \quad (41)$$

$$EV^*\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_{-\infty}^0 \frac{\sum_{a \in \pi} SLAVR(TLM_FQL)(a)}{|\Pi|} + \int_0^{+\infty} \frac{\sum_{a \in \pi} SLAVR(TLM_FQL)(a)}{|\Pi|} \quad (42)$$

$$EV^*\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong Low \rightarrow EV\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong Low \quad (43)$$

$$EV^*\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T\right) \cong Low \rightarrow EV\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T\right) \cong Low \quad (44)$$

$$EV^*\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong Low \geq EV\left(\frac{SLAVR(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong Low \quad (45)$$

EM4: Scalability (SC(TLM_FQL)): The expected value of the scalability of the TLM_FQL is significantly higher compared to the expected value of the scalability of the TLM_FQL in small scenario.

$$\begin{aligned} EV^*\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T\right) &= \int_{-\infty}^0 \frac{\frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i) - MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i}}{|\Pi|} \\ &+ \int_0^{+\infty} \frac{\frac{MK(\sum_{i=1}^m T_i + \sum_{i=1}^m \Delta T_i) - MK(\sum_{i=1}^m T_i)}{\sum_{i=1}^m \Delta T_i}}{|\Pi|} \end{aligned} \quad (46)$$

$$EV^*\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_{-\infty}^0 \frac{\sum_{a \in \pi} SC(TLM_FQL)(a)}{|\Pi|} + \int_0^{+\infty} \frac{\sum_{a \in \pi} SC(TLM_FQL)(a)}{|\Pi|} \quad (47)$$

$$EV^*\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong High \geq EV\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong High \quad (48)$$

$$EV^*\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T\right) \cong High \rightarrow EV\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T\right) \cong High \quad (49)$$

$$EV^*\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong High \geq EV\left(\frac{SC(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong High \quad (50)$$

EM5: Reliability (RE(TLM_FQL)): The expected value of the reliability of the TLM_FQL is significantly less compared to the expected value of the reliability of the TLM_FQL in small scenario.

$$EV^*\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T\right) = \int_{-\infty}^0 \sum_{j=1}^{j=n} \frac{\frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)}}{|\Pi|} + \int_0^{+\infty} \sum_{j=1}^{j=n} \frac{\frac{\sum_{i=1}^m Ex_time(T_i)}{Av_time(FN_j)}}{|\Pi|} \quad (51)$$

$$EV^*\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T\right) = \sum_{d \in D} d \int_{-\infty}^0 \frac{\sum_{a \in \pi} RE(TLM_FQL)(a)}{|\Pi|} + \int_0^{+\infty} \frac{\sum_{a \in \pi} RE(TLM_FQL)(a)}{|\Pi|} \quad (52)$$

$$EV^*\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong High \geq EV\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T - \alpha\right) \cong High \quad (53)$$

$$EV^*\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T\right) \cong High \cong EV\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T\right) \cong High \quad (54)$$

$$EV^*\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong High \geq EV\left(\frac{RE(TLM_FQL)}{\Pi}, \Delta T + \alpha\right) \cong High \quad (55)$$

6. Results and Discussion

The proposed TLM_FQL framework is compared with three of the recent existing works, i.e., DDPG [17], DCS [20], HHO [19], and FQL [21]. The task scheduling environment is composed of two layers: the IoT devices layer and fog layer. The IoT devices layer is responsible for generating the tasks which follow a stochastic arrival pattern. Similarly, the fog layer is composed of multiple fog nodes with varying processing capacity which process the tasks which are latency insensitive. If sufficient resources are not available, then they will be processed at the cloud layer with an unlimited number of resources. The TLM_FQL framework interacts with the fog environment based on the current state of operation. The current state is determined by considering the current dynamics of tasks, resources, and network conditions. The local task scheduling actions are chosen by the

FQL considering the feedback of the TLM. The fog environment updates its state after the action and provides feedback over the evaluation metrics.

For simulation purposes Simcan2Fog simulator is used which is a recent extension of the fog computing simulator [26,27]. It incorporates the real-time behavior of the network, hardware modeling, and cloud orchestration. It allows precise representation of network behavior considering topology management, obstacles detection, transport protocol, and traffic control mechanism. The Simcan2Fog simulator is an extension of the Simcan2Cloud simulator which provides higher end simulation features like virtualization, data center operations, queuing mechanism, and cost evaluations. The simulation parameters initialization description is provided in Table 3.

Table 3. Simulation parameters initialization.

Tiny Language Model	
Number of parameters	0.10 billion
Model size	0.277 GiB
Accuracy	98.44%.
Small Scenario	
Task arrival pattern	Poisson
Tasks	10–100
Sensors	10–100
Actuators	10–100
Controllers	5–100
Fog nodes	10–100
Data centers	10–100
Resident memory	117.86 MiB
Execution time	0.08 ms
Simulation time	3600 s
Relationship between simulated time and wall clock time	38.5 units
Number of processed events	18,976
Processed events/Sec	191,117.35
Big Scenario	
Tasks	1000–10,000
Sensors	1000–10,000
Actuators	1000–10,000
Controllers	51–80
Fog nodes	1000–10,000
Data centers	1000–10,000
Resident memory	646.86 MiB
Execution time	81.89 ms
Simulation time	3600 s
Relationship between simulated time and wall clock time	44.08 units
Number of processed events	28,399,687
Processed events/Sec	335,430.89

Table 3. *Cont.*

FQL	
Task arrival pattern	Poisson
Flow type	Conditional normalizing flow
Step size	0.08–1.0
Network depth	3–4
Hidden units	300
Activation	ReLU
DDPG	
Task arrival pattern	Poisson
Policy network	1×10^{-6}
Critic learning rate	1×10^{-5}
Architecture	2*300 Neurons
Activation function	tanh
Target update	0.005
Exploration noise	0.5
DCS	
Task arrival pattern	Poisson
Population size	30–45
Maximum generations	300–600
Levy flight step	Discrete Levy
Step size	1
Discovery probability	0.3–0.5
HHO	
Population size	30
Maximum generations	300–600
Energy decay	Linear
Random jump strength	[0, 2]
Search strategy	Exploration and exploitation

6.1. Small Scenario

The small scenario is localized and has a limited time frame which operates with a few tasks, sensors, actuators, controllers, fog nodes, and data centers.

EM1: Cost (CO(TLM_FQL)):

A graph of the number of fog nodes versus cost in the small scenario is shown in Figure 3. It is observed from the graph that the cost of TLM_FQL remained to be consistently less with the increase in the number of fog nodes, because its ability to handle complex and multimodal action space is well developed which yields superior performance. The cost of FQL kept increasing with the increase in the number of fog nodes as the Q value estimation needed to be performed at every stage of training. The cost of DDPG is found to be moderate with the increase in the number of fog nodes, because it can handle the continuous state space environment, but it is sensitive towards hyperparameter initialization. The cost of DCS and HHO are found to be very high with the increase in the number of fog nodes, because of poor state space exploration and using random parameters during exploration stage which become trapped in local minima.

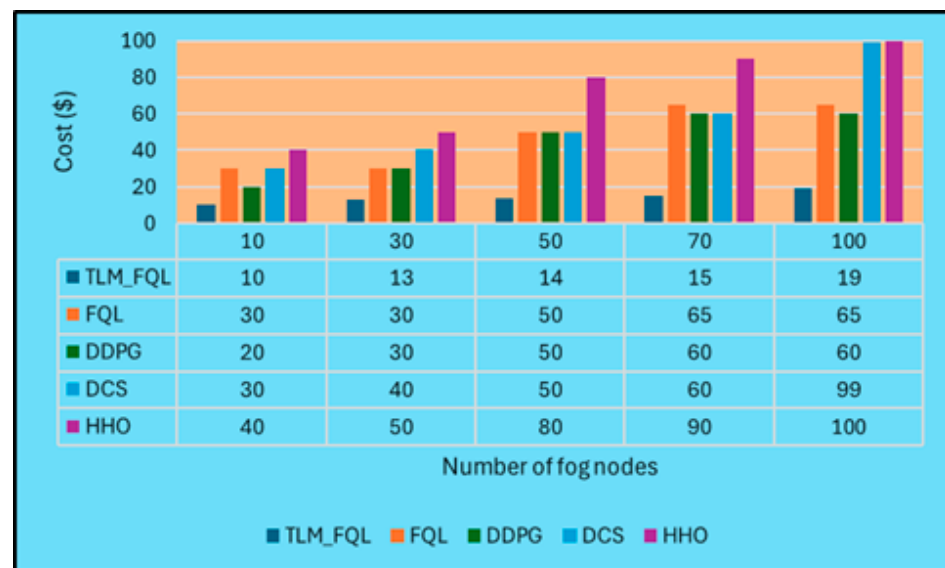


Figure 3. Number of fog nodes versus cost in small scenario.

EM2: Energy consumption (EC(TLM_FQL):

A graph of the number of tasks versus energy consumption in the small scenario is shown in Figure 4. It is observed from the graph that the energy consumption of TLM_FQL remained consistently low with the increase in the number of incoming tasks, as it can adapt effectively to changing dynamics of the network without the need for complex fine tuning of noise. The energy consumption of FQL is very high over the number of tasks because it becomes trapped in double instability and the error in the Q value estimation propagates through the transmission of the flow policy. The energy consumption of DDPG is found to be moderate over the increase in the number of tasks, as it is subjected towards overestimation of the Q values leading to suboptimal policies. The energy consumption of DCS and HHO are found to be very high with the increase in the number of tasks, as it demands frequent modification of core operators and poor prioritization of tasks, resulting in conflicting goals.



Figure 4. Number of tasks versus energy consumption in small scenario.

EM3: Service Level Agreement (SLA) violation rate (SLAVR(TLM_FQL)):

A graph of the number of tasks versus SLA violation rate in the small scenario is shown in Figure 5. The SLA violation rate of TLM_FQL remained low with the increase in the number of tasks, because it can be directly fine-tuned for both offline and online rollouts. The SLA violation rate is high for FQL as learning signals weaken due to the diffusion of temporal difference error across the flow signals. The SLA violation of DDPG and DCS kept increasing with the increase in the number of tasks, because the policies are deterministic which leads to poor sampling efficiency and suboptimal solutions. The SLA violation of HHO is found to be moderate over the increase in the number of tasks due to its inability to perform a global search and inconsistency in the behavior at the individual and group level of operation.

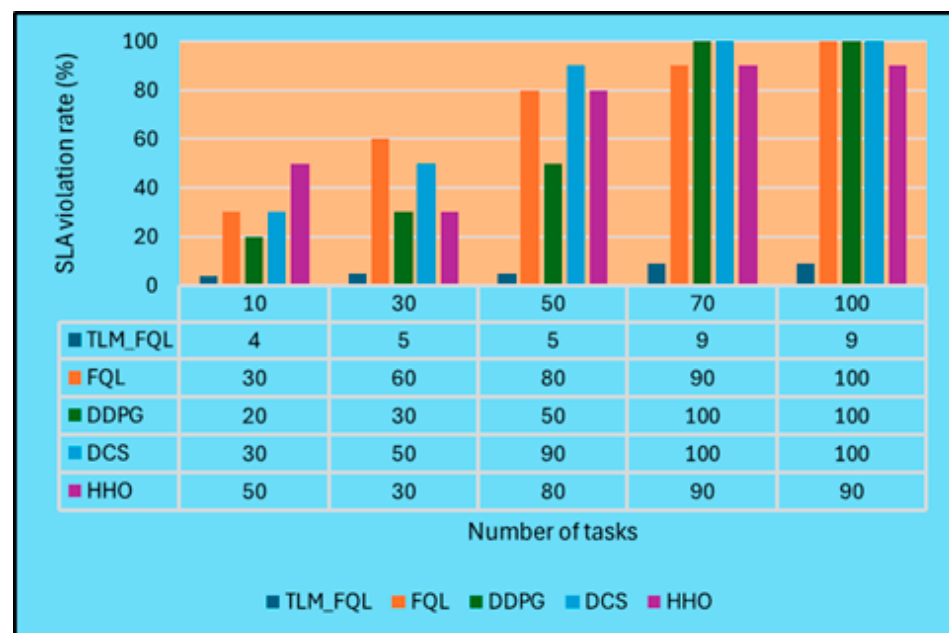


Figure 5. Number of tasks versus SLA violation rate in small scenario.

EM4: Scalability (SC(TLM_FQL)):

A graph of the number of tasks versus scale up in the small scenario is shown in Figure 6. It is observed from the graph that the TLM_FQL scales up easily with the increase in the number of tasks, as its scaling capacity is well developed since it used flow matching for training and avoids the accumulation of bias over the horizon. The scale up capability of FQL is less as the flow updated and is misguided due to the interference of errors in the Q function at the early stage of learning. The scale up capability of the DDPG and HHO are found to be poor with the increase in the number of tasks, because of unstable updates of Q values leading to potential divergence towards function approximation. The scale up capability of DCS is found to be moderate with the increase in the number of tasks, as it becomes trapped in local minima and exhibits premature convergence without effectively exploring the state space.

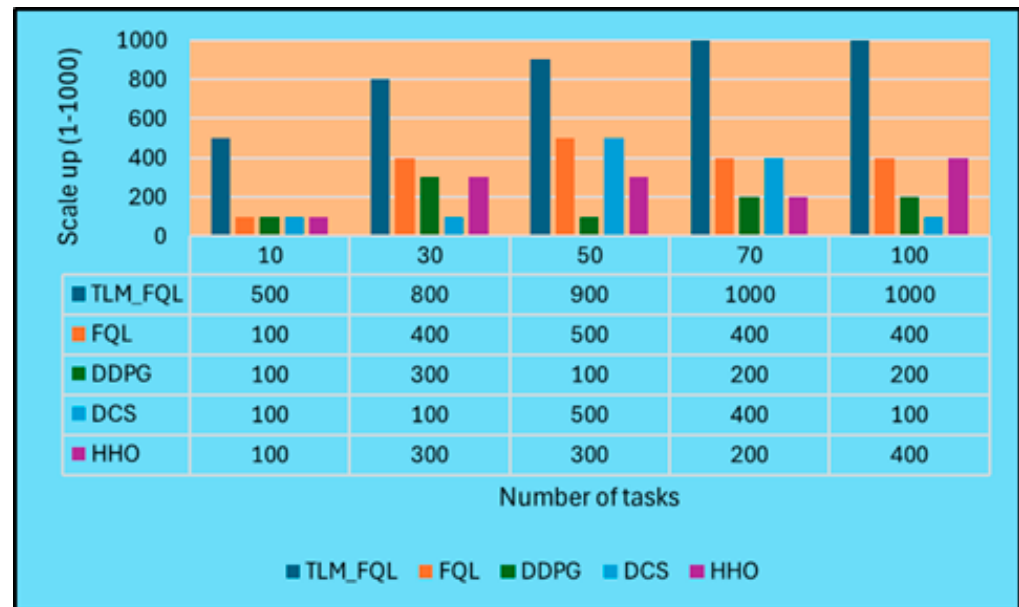


Figure 6. Number of tasks versus scale up in small scenario.

EM5: Reliability (RE(TLM_FQL)):

A graph of the time versus failure rate in the small scenario is shown in Figure 7. It is observed from the graph that the failure rate of TLM_FQL is consistently less over time, as it effectively handles the training instability by using the one-step policy and policy decoupling strategy. The failure rate of FQL is high over the time as the optimal exploration of the state space is not guaranteed due to the involvement of latent space variables during exploration. The failure rate of DDPG and DCS are very high over time, as they find it difficult to adapt to discrete space from continuous state space environment. The failure rate of HHO is moderate over time. Due to the application of diversified hunting strategies, the technique is capable enough of assuring the optimal solution despite the changes in the fog nodes operating conditions.

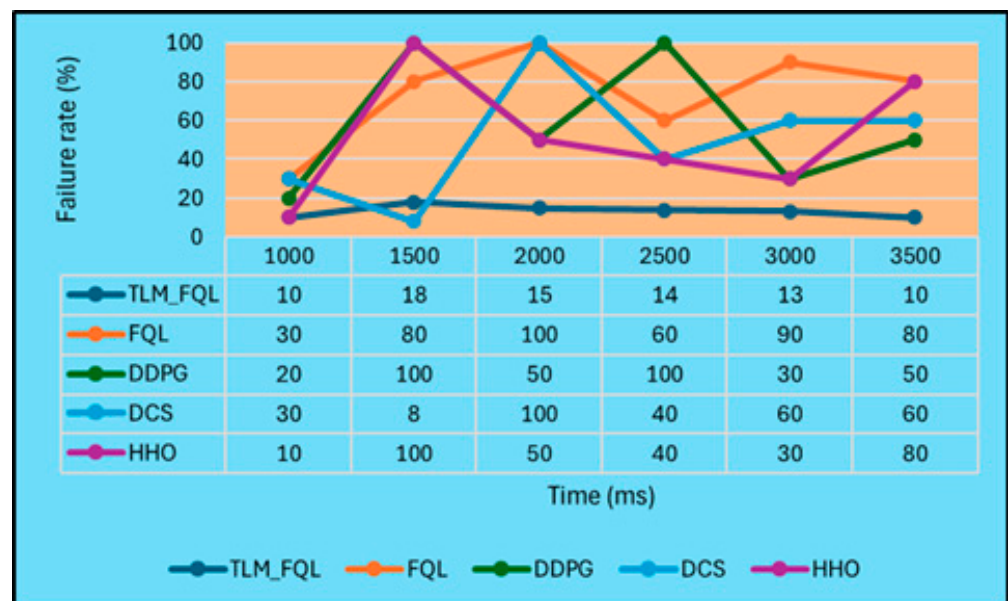


Figure 7. Time versus failure rate in small scenario.

6.2. Big Scenario

The big scenario is expansive and spans a larger time which operates with a big number of tasks, sensors, actuators, controllers, fog nodes, and data centers.

EM1: Cost (CO(TLM_FQL)):

A graph of the number of fog nodes versus cost in the big scenario is shown in Figure 8. It is observed from the graph that the cost of TLM_FQL is consistently less with the increase in the number of fog nodes, because the one-step action process is faster and avoids the need for high computational expenses for sampling and backpropagation. The cost of FQL is high towards the increasing number of fog nodes as the Q learning formulate outdated policies due to a slow understanding of fog nodes mobility and frequent failure of fog nodes. The cost of DDPG is found to be moderate with the increase in the number of fog nodes. Higher computation cost is incurred because it demands more memory and resources during forward passes of evaluation. The cost of DCS and HHO are found to be very high with the increase in the number of fog nodes, as they do long-time calculations and demand a significant number of resources to complete the execution.

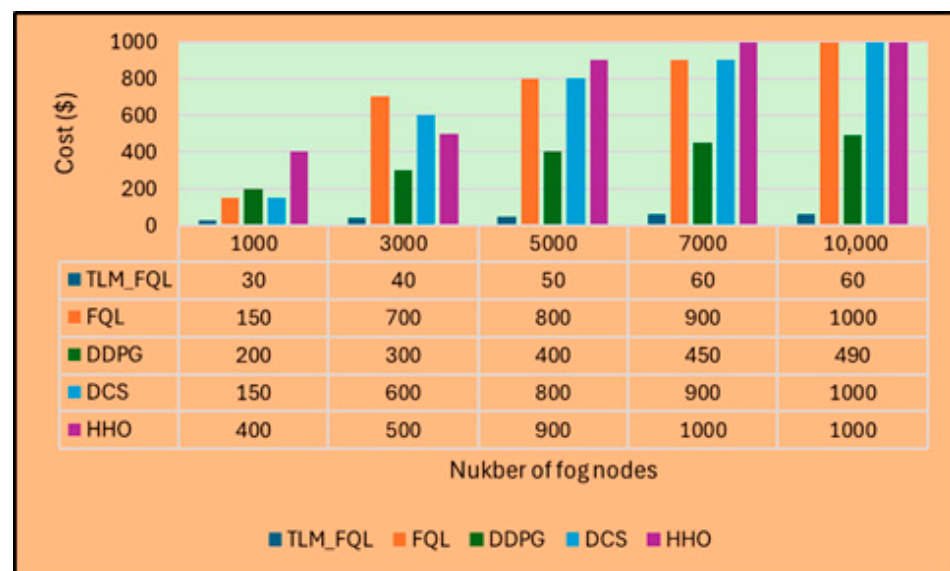


Figure 8. Number of fog nodes versus cost in big scenario.

EM2: Energy consumption (EC(TLM_FQL)):

A graph of the number of tasks versus energy consumption in the big scenario is shown in Figure 9. It is observed from the graph that the energy consumption of TLM_FQL is consistently low with an increase in the number of tasks, as it prevents repeated recursive backpropagation through a multi-step sampling process. The energy consumption of FQL is high over the increase in the number of tasks as the flow transformations demand a large replay buffer for action storage and require diverse state space coverage to generalize the policies. The energy consumption of DDPG and HHO are found to be more with the increase in the number of tasks, as they suffer from an unstable optimization effect and are easily trapped into stagnation. The energy consumption of DCS is moderate with the increase in the number of tasks, because it has high computational complexity and struggles to adapt to a discrete problem state space environment.

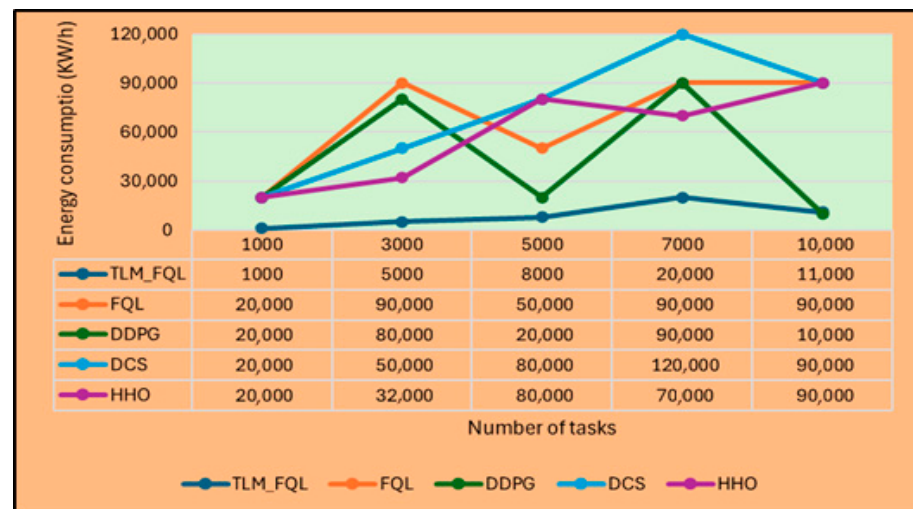


Figure 9. Number of tasks versus energy consumption in big scenario.

EM3: Service Level Agreement (SLA) violation rate (SLAVR(TLM_FQL)):

A graph of the number of tasks versus SLA violation rate in the big scenario is shown in Figure 10. It is observed from the graph that the SLA violation rate of TLM_FQL is consistently less with an increase in the number of tasks. It is capable enough of achieving faster inference through ordinary deterministic differential equations. The SLA violation rate of FQL is high due to the generation of biased policies because of poor exploration of the state space environment. The SLA violation rate of DDPG is found to be moderate with the increase in the number of tasks, as it struggles to learn whenever subjected to sparse rewards. The SLA violation rate of DCS and HHO are found to be very high with the increase in the number of tasks. The initialization phase is in a randomized order which limits the diversity of solutions and results in local optima solutions.

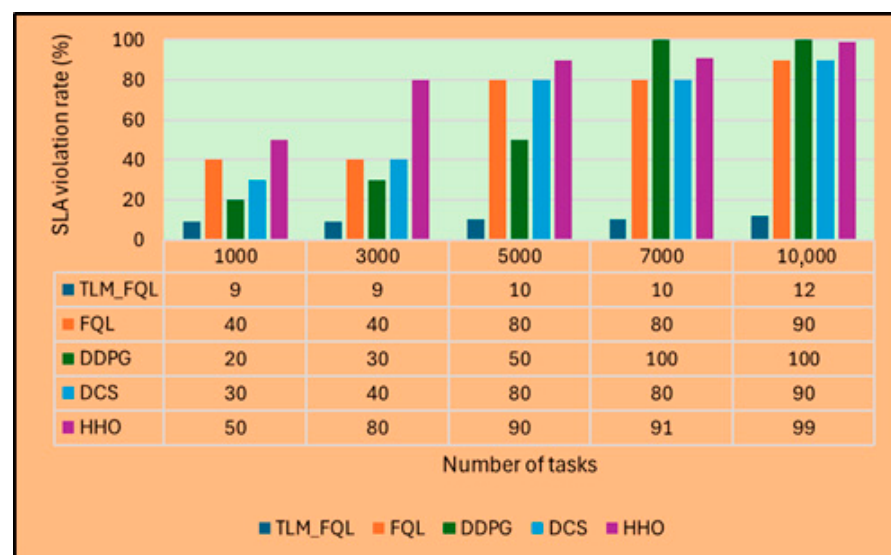


Figure 10. Number of tasks versus SLA violation rate in big scenario.

EM4: Scalability (SC(TLM_FQL)):

A graph of the number of tasks versus scale up in the big scenario is shown in Figure 11. It is observed from the graph that the scalability of TLM_FQL is found to be very high with the increase in the number of tasks, as it avoids the need for backpropagation through multiple steps of the generation process. The scalability of FQL is less due to poor tuning of sensitive parameters like the discount factor, number of flow steps, and target updating

rate. The scalability of DCS and HHO are found to be moderate with the increase in the number of tasks, as they are computationally expensive and demand complex conversion between continuous solutions and discrete solutions. The scalability of DDPG is found to be very low with the increase in the number of tasks. The inaccurate feedback leads to frequent reinforcement of suboptimal actions.

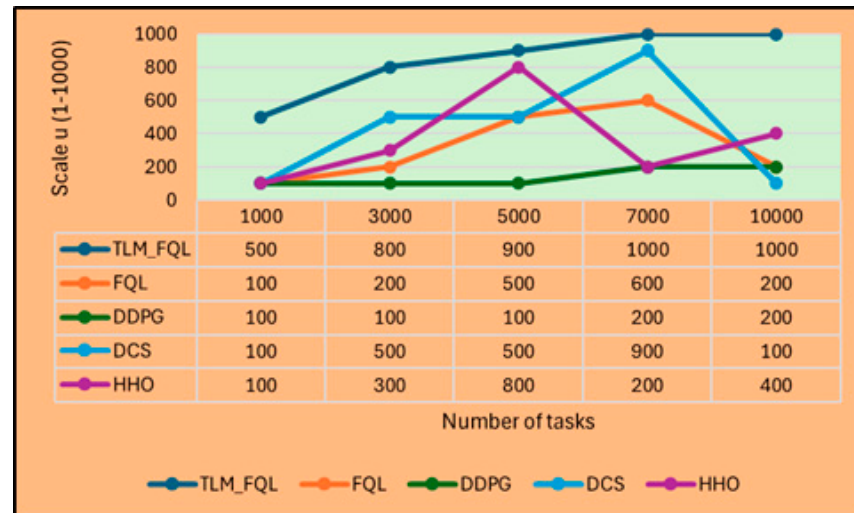


Figure 11. Number of tasks versus scale up in big scenario.

EM5: Reliability (RE(TLM_FQL)):

A graph of time versus failure rate in the big scenario is shown in Figure 12. It is observed from the graph that the failure rate of TLM_FQL remained consistently less with respect to time. The flow policy is trained using the behavior cloning mechanism and achieves the maximum value function using the one-step policy. The failure rate of FQL is high as the constraints for task scheduling is handled only through penalty rewards and demands frequent model retraining with the increase in the heterogeneity of the fog nodes and tasks. The failure rate of DDPG and DCS are found to be moderate over time due to the poor balance between exploration and exploitation, with tendency towards premature convergence being high. The failure rate of HHO is found to be very high over time due to poor search ability and a chaotic optimization strategy causing negative impact over the exploration capability of the state space which destabilizes the functioning of the algorithm.

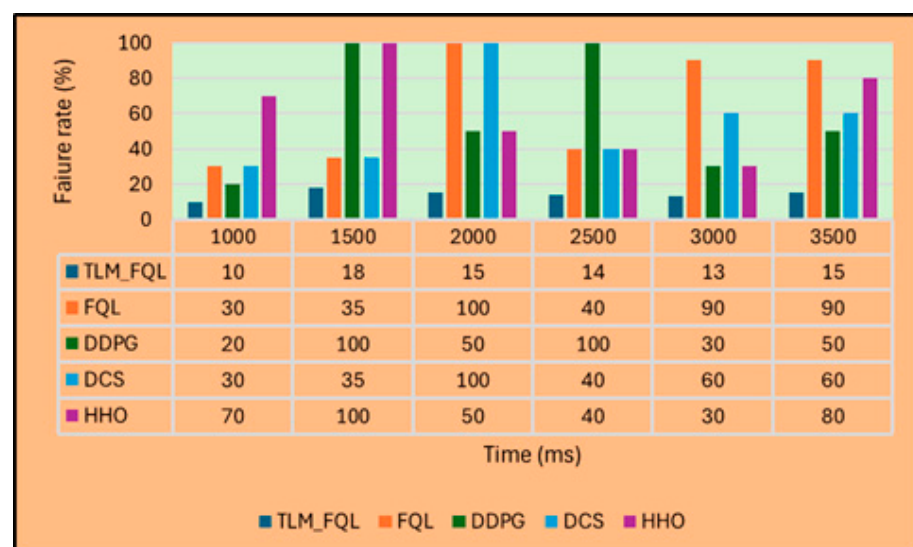


Figure 12. Time versus failure rate in big scenario.

7. Conclusions

The paper presents a novel Tiny Language Model-inspired flow Q learning framework for task scheduling in fog computing. The heuristics of TLM are provided as guidance for Q learning to perform flow models and behavior cloning of policies through one-step optimization. The structure priors of TLM reduce the tendency of overestimation of the Q value and make the bootstrapping process smoother. Generalization of the behavior cloning is easy through demonstrations and lowers the risks of hallucinations. The experimental results demonstrate that the performance of the TLM_FQL is good with respect to metrics like cost, energy consumption, SLA violation, and scalability considering small and big fog computing scenarios. The results are further validated using expected value analysis and results are found to be satisfactory. For future work, trajectory will be used to properly represent the state, action, and reward of the learning agent. Pruning of unsafe trajectories will be carried out to enhance the robustness of the scheduling policies and improve learning efficiency. The noise in the state space is removed through selective experience replay and produces policy reliability under adverse operating conditions. The integration of safe reinforcement learning along with the trajectory pruning ensure faster learning and prevents the catastrophic failures of the tasks.

Author Contributions: Conceptualization, B.K. and S.G.S.; methodology, B.K.; software, S.G.S.; validation, B.K. and S.G.S.; formal analysis, B.K.; investigation, S.G.S.; resources, B.K.; data curation, B.K.; writing—original draft preparation, B.K.; writing—review and editing, S.G.S.; visualization, B.K.; supervision, S.G.S.; project administration, B.K.; funding acquisition, S.G.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed at the corresponding authors.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Refonaa, J.; Maheswari, M.; Poornima, D.; Shabu, S.J.; Gowri, M.; Praveen, S.; Amshavalli, R.S. Edge and Fog Computing for Sustainability. In *Energy Efficient Algorithms and Green Data Centers for Sustainable Computing*; IGI Global Scientific Publishing: Hershey, PA, USA, 2025; pp. 79–108.
2. Chuan, W.C.; Laghari, S.U.A.; Manickam, S.; Ashraf, E.; Karuppayah, S. Challenges and Opportunities in Fog Computing Scheduling: A Literature Review. *IEEE Access* **2025**, *13*, 14702–14726. [[CrossRef](#)]
3. Rudner, T.G.; Pan, X.; Li, Y.L.; Shwartz-Ziv, R.; Wilson, A.G. Fine-Tuning with Uncertainty-Aware Priors Makes Vision and Language Foundation Models More Reliable. In Proceedings of the ICML 2024 Workshop on Structured Probabilistic Inference and Generative Modeling, Vienna, Austria, 26 July 2024.
4. Fakour, F.; Mosleh, A.; Ramezani, R. A structured review of literature on uncertainty in machine learning & deep learning. *arXiv* **2024**, arXiv:2406.00332. [[CrossRef](#)]
5. Aldossary, D.; Aldahasi, E.; Balharith, T.; Helmy, T. A Systematic Literature Review on Load-Balancing Techniques in Fog Computing: Architectures, Strategies, and Emerging Trends. *Computers* **2025**, *14*, 217. [[CrossRef](#)]
6. Nemati, A.M.; Mansouri, N. Resource allocation in fog computing: A survey on current state and research challenges. *Knowl. Inf. Syst.* **2025**, *67*, 2091–2170. [[CrossRef](#)]
7. Sauret, A.; Cerin, C.; Diaz, G.; Rivalan, J.; Menouer, T.; Boussetta, K. A Survey on Dynamic SLAs for Fog Computing. *Authorea Preprints* **2025**. [[CrossRef](#)]
8. Toghyani, M.; Khorsand, R.; Khaksar, H. QoS-SLA-aware optimization framework for IoT-Service placement in integrated Fog-Cloud computing. *J. Grid Comput.* **2025**, *23*, 1.

9. Wang, B.; Tao, B.; Jing, H.; Dou, H.; Wang, Z. Controllable Flow Matching for Online Reinforcement Learning. *arXiv* **2025**, arXiv:2511.06816. [[CrossRef](#)]
10. Pfrommer, S.; Huang, Y.; Sojoudi, S. Reinforcement learning for flow-matching policies. *arXiv* **2025**, arXiv:2507.15073. [[CrossRef](#)]
11. Gross, R.D.; Tzach, Y.; Halevi, T.; Koresh, E.; Kanter, I. Tiny language models. *Phys. A Stat. Mech. Its Appl.* **2025**, *681*, 131102.
12. Lamaakal, I.; Maleh, Y.; El Makkaoui, K.; Ouahbi, I.; Pławiak, P.; Alfarraj, O.; Almousa, M.; El-Latif, A.A.A. Tiny language models for automation and control: Overview, potential applications, and future research directions. *Sensors* **2025**, *25*, 1318. [[CrossRef](#)]
13. Hong, J.; Dragan, A.; Levine, S. Q-sft: Q-learning for language models via supervised fine-tuning. *arXiv* **2024**, arXiv:2411.05193.
14. Hillier, D.; Guertler, L.; Tan, C.; Agrawal, P.; Ruirui, C.; Cheng, B. Super tiny language models. *arXiv* **2024**, arXiv:2405.14159. [[CrossRef](#)]
15. Ai, Y.-Q.; Hu, S.; Liu, S.; Jui, S.; Wang, Y.; Tang, Y.; Han, K.; Liu, F.; Yu, Y. Rethinking optimization and architecture for tiny language models. In Proceedings of the Forty-First International Conference on Machine Learning, Vienna, Austria, 21–27 July 2024.
16. Choppara, P.; Mangalampalli, S.S. Adaptive Task Scheduling in Fog Computing Using Federated DQN and K-Means Clustering. *IEEE Access* **2025**, *13*, 75466–75492. [[CrossRef](#)]
17. Choppara, P.; Mangalampalli, S.S. Resource Adaptive Automated Task Scheduling Using Deep Deterministic Policy Gradient in Fog Computing. *IEEE Access* **2025**, *13*, 25969–25994. [[CrossRef](#)]
18. Mangalampalli, S.S.; Reddy, P.V.; Reddy Karri, G.; Tippani, G.; Kota, H. Priority-Aware Multi-Objective Task Scheduling in Fog Computing Using Simulated Annealing. *Sensors* **2025**, *25*, 5744. [[CrossRef](#)]
19. Ali, A.; Shah, S.A.A.; Al Shloul, T.; Assam, M.; Ghadi, Y.Y.; Lim, S.; Zia, A. Multiobjective Harris Hawks optimization-based task scheduling in cloud-fog computing. *IEEE Internet Things J.* **2024**, *11*, 24334–24352. [[CrossRef](#)]
20. Hosseini, S.M.; Shirvani, M.H.; Motameni, H. Multi-objective discrete Cuckoo search algorithm for optimization of bag-of-tasks scheduling in fog computing environment. *Comput. Electr. Eng.* **2024**, *119*, 109480. [[CrossRef](#)]
21. Park, S.; Li, Q.; Levine, S. Flow q-learning. *arXiv* **2025**, arXiv:2502.02538. [[CrossRef](#)]
22. Yang, F.; Liu, J.; Li, K. LLM-Guided Reinforcement Learning for Interactive Environments. *Mathematics* **2025**, *13*, 1932. [[CrossRef](#)]
23. Misirli, J.; Casalicchio, E. An analysis of methods and metrics for task scheduling in fog computing. *Future Internet* **2023**, *16*, 16. [[CrossRef](#)]
24. Chakraborty, R.; Ahmadkhani, S.; Świerczyński, J.; Boguslawski, K.; Tecmer, P. Expectation Value-pCCD-Based Methods for Single-Electron Properties. *J. Phys. Chem. A* **2025**, *129*, 6713–6732. [[CrossRef](#)]
25. Hashemi-Tabatabaei, M.; Amiri, M.; Keshavarz-Ghorabae, M. An Expected Value-Based Symmetric–Asymmetric Polygonal Fuzzy Z-MCDM Framework for Sustainable–Smart Supplier Evaluation. *Information* **2025**, *16*, 187. [[CrossRef](#)]
26. de Aguilar, U.; Cañizares, P.C.; Núñez, A. Simcan2Fog: A discrete-event platform for the modelling and simulation of Fog computing environments. *SoftwareX* **2025**, *32*, 102424. [[CrossRef](#)]
27. Ghaseminya, M.M.; Fazeli, S.A.S.; Abouei, J.; Abbasi, E. Fogfaas: Providing serverless computing simulation for iFogSim and edge cloud. *J. Supercomput.* **2025**, *81*, 666. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.